Chair of Network Architectures and Services
School of Computation, Information, and Technology
Technical University of Munich

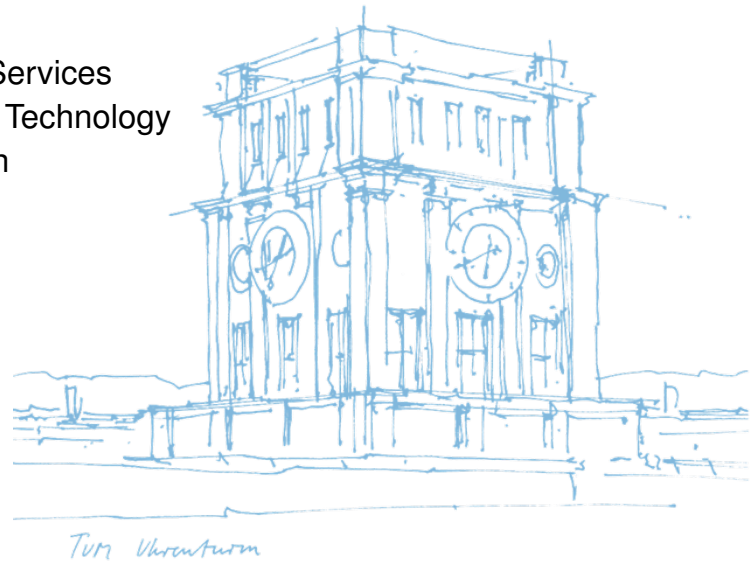
QUIC Improvements Using XDP

Final talk for the Master's Thesis by

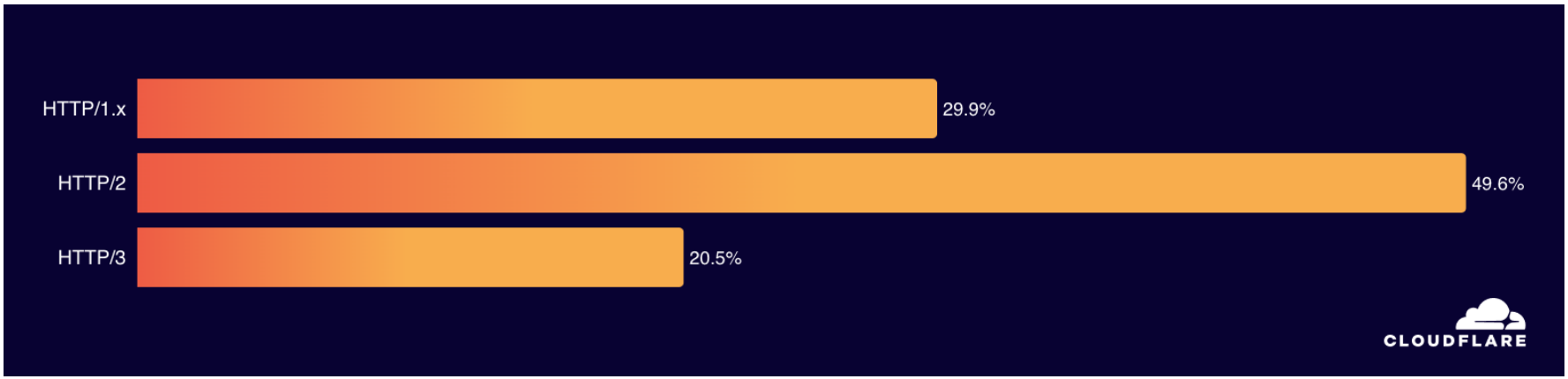
Dominik Ritzenhoff
advised by Johannes Späth, Marcel Kempf

Monday 18th August, 2025

Chair of Network Architectures and Services
School of Computation, Information, and Technology
Technical University of Munich



Motivation



- Roughly 20% of the world's internet traffic now runs on HTTP/3, the protocol built on QUIC[1]
- QUIC throughput is not able to match TCP with increasing bandwidth[2]
- Packet I/O is the largest bottleneck for major QUIC implementations in terms of its maximum achievable goodput[3]
- Work has been done to address this bottleneck: DPDK, Linux-in-QUIC, and Generic Receive Offload (GRO) with Generic Segmentation Offload (GSO).

XDP offers an additional approach

[1] https://radar.cloudflare.com/year-in-review/2024#http-versions

[2] Zhang et al., Quic is not quick enough over fast internet, WWW '24, pp. 2713–2722

[3] Jaeger et al., Quic on the highway, June 2023, pp. 1–9.

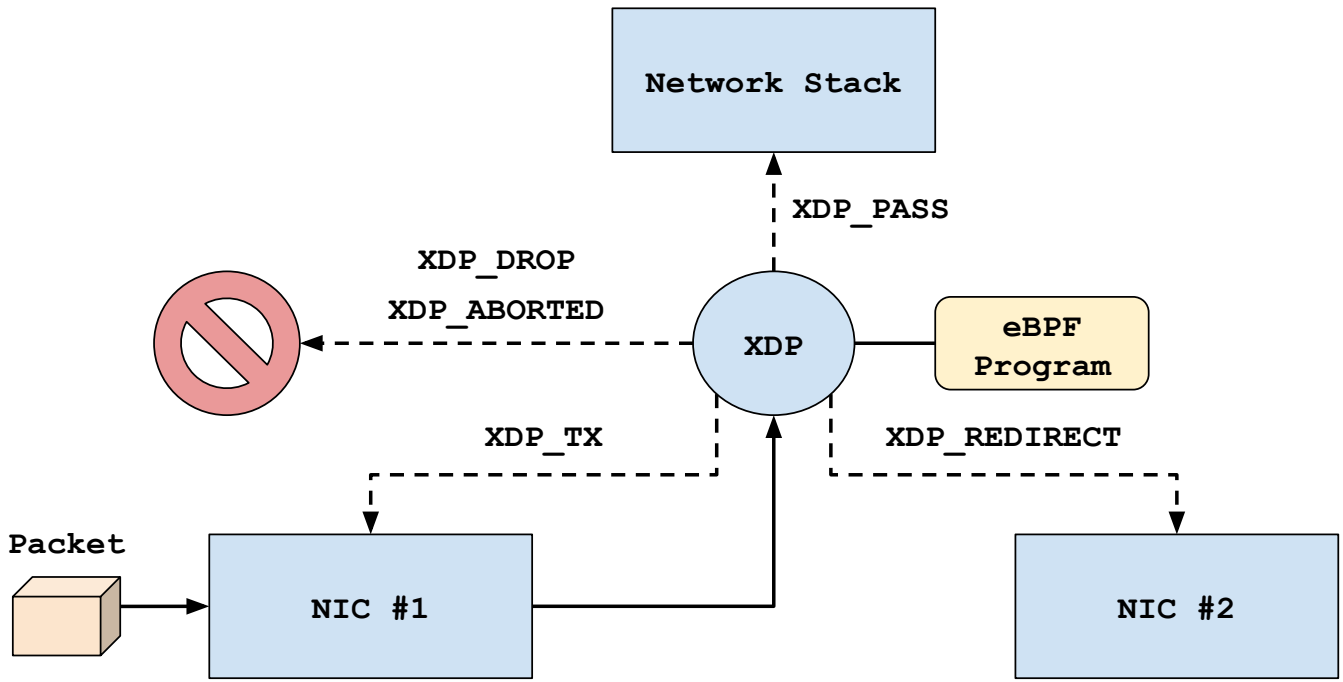
Research Questions

- To what extent can we increase QUIC performance by using XDP in a trivial way? Is there optimization potential?

- Can a taxonomy be created for QUIC optimizations, categorized by:
  - Implementation complexity
  - Performance improvement potential
  - Unwanted side effects
  - Privilege level usage
  - Deployment

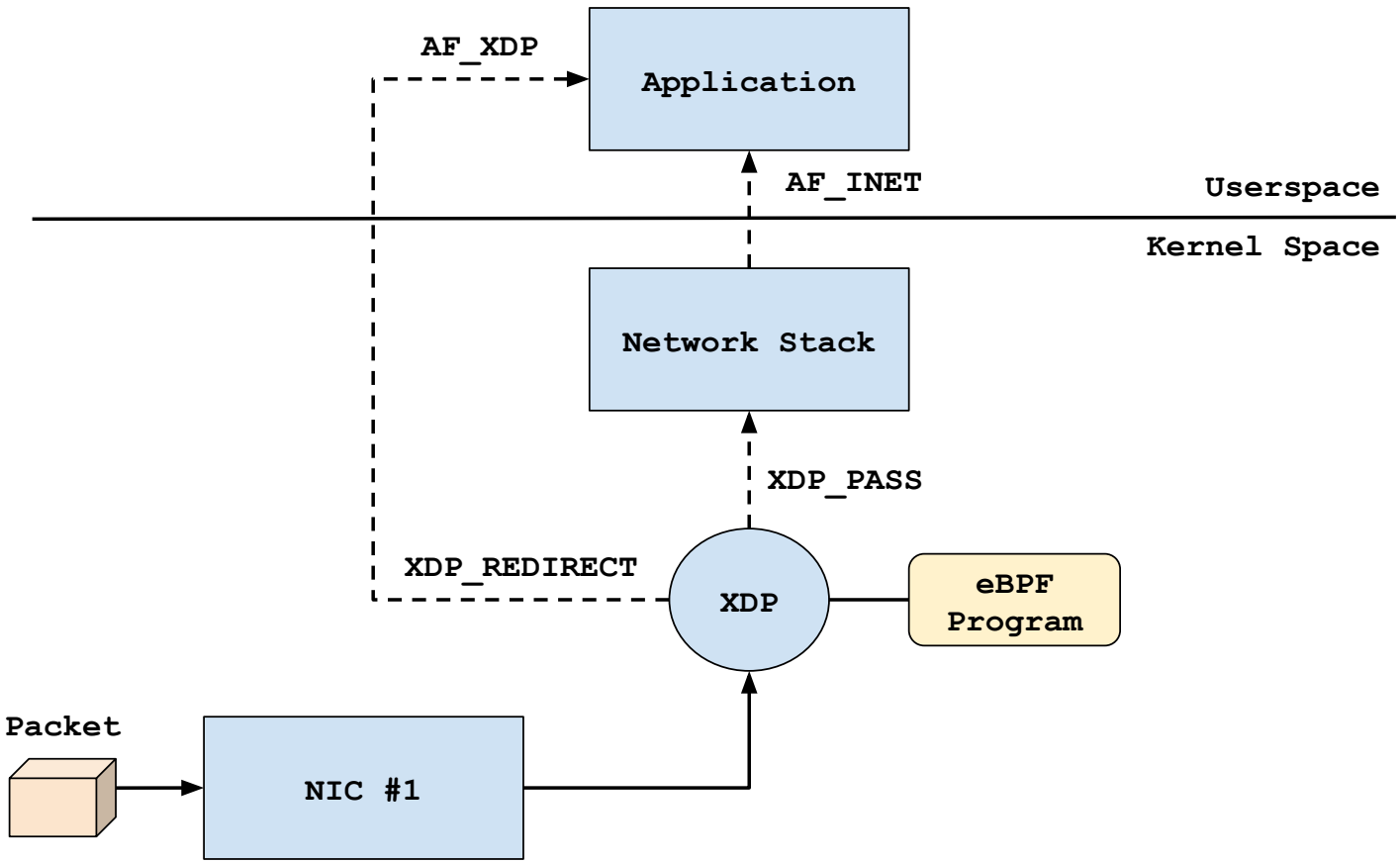
XDP Primer

- XDP is a native Linux hook point for eBPF bytecode to run in
- Decide on an action for each packet: `XDP_DROP`, `XDP_PASS`, `XDP_REDIRECT`, `XDP_ABORTED`, `XDP_TX`



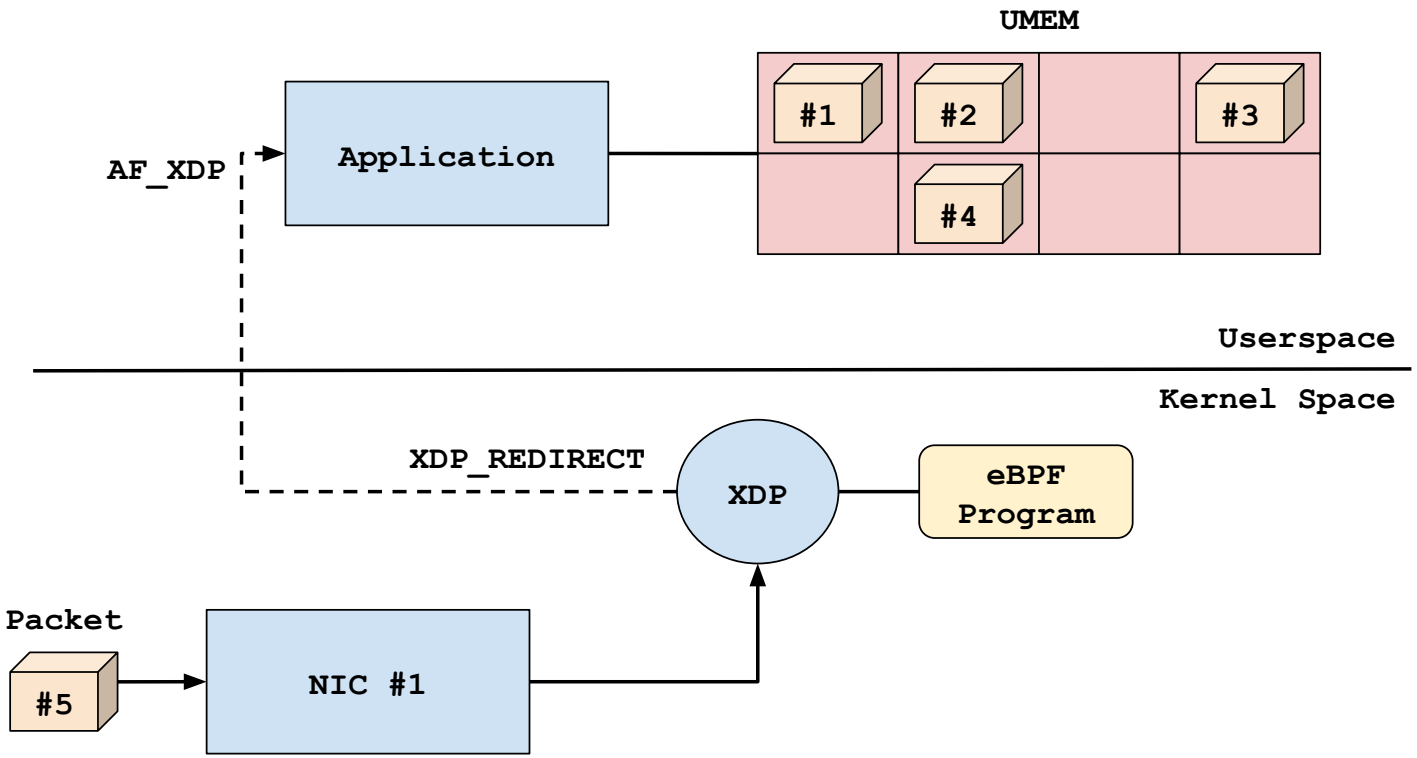
XDP Primer

- AF_XDP is an address family in Linux similar to AF_INET or AF_INET6
- Works with XDP to bypass the network stack
- Helps avoid costly per packet processing



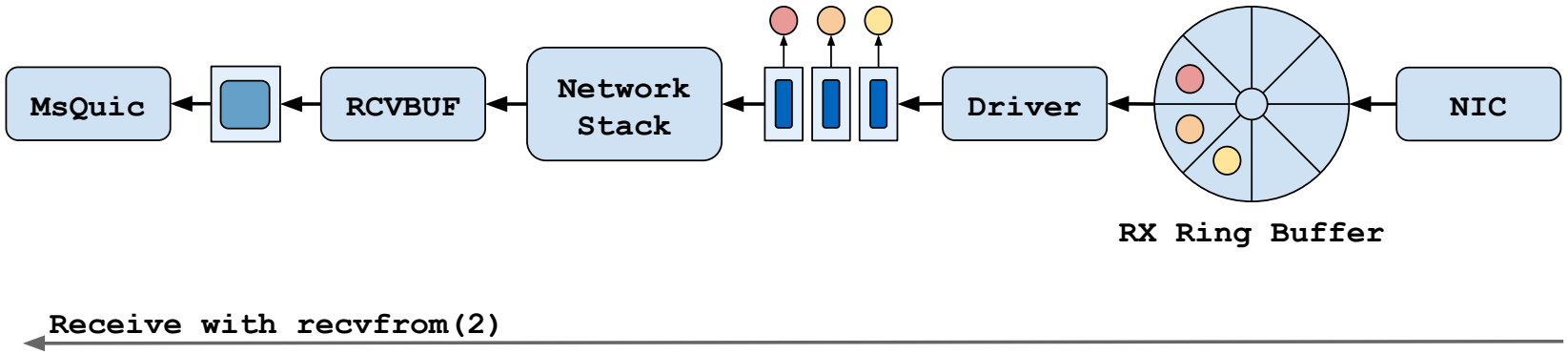
AF_XDP Primer

- UMEM is a data buffer in userspace memory; it stores packets
- Packets be directly copied into the UMEM from the NIC (i.e. zero-copy or zc)
- Alternatively, packets can be copied into kernel memory first and later into the UMEM



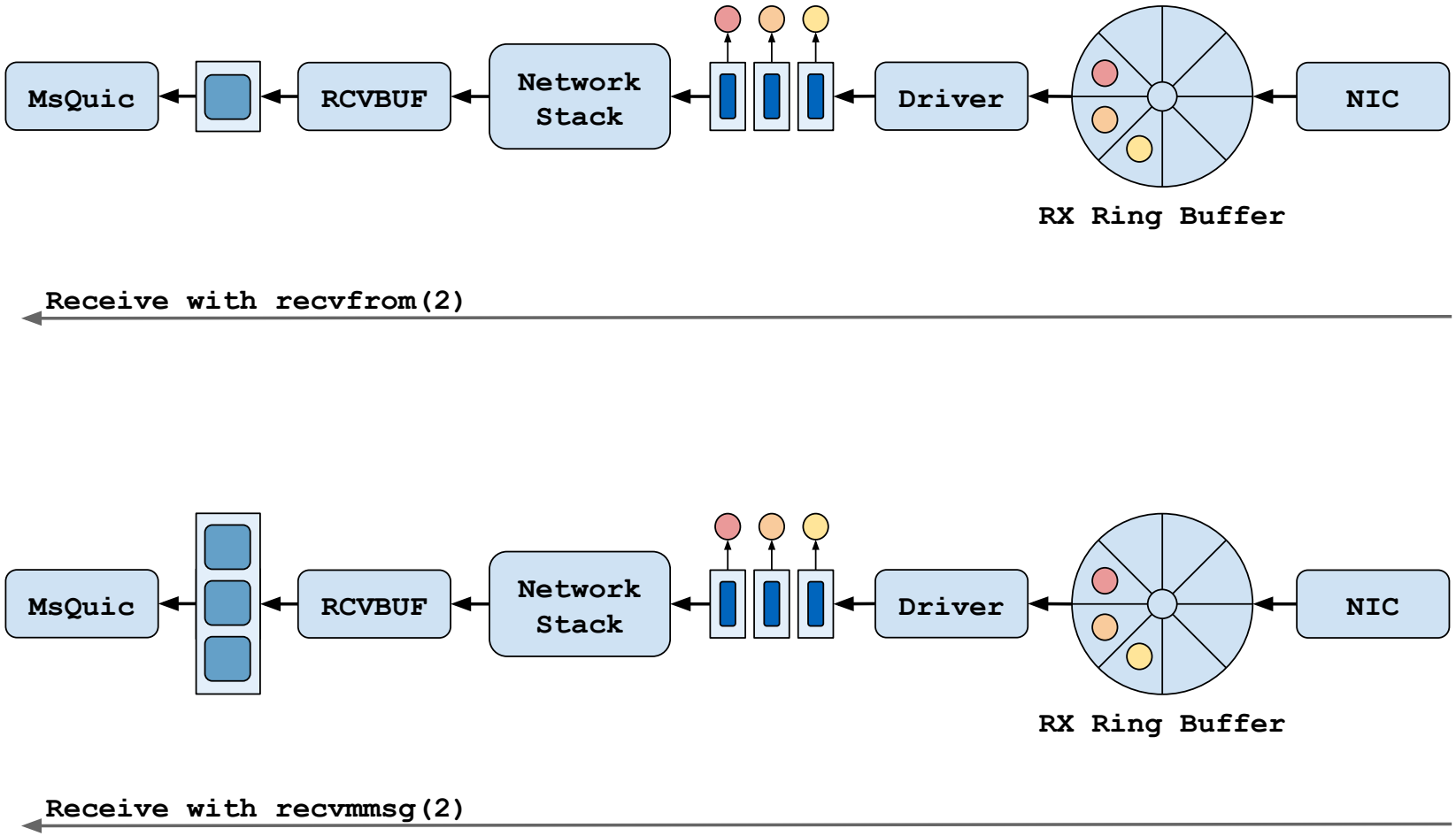
Additional Optimizations

recvmmsg(2)



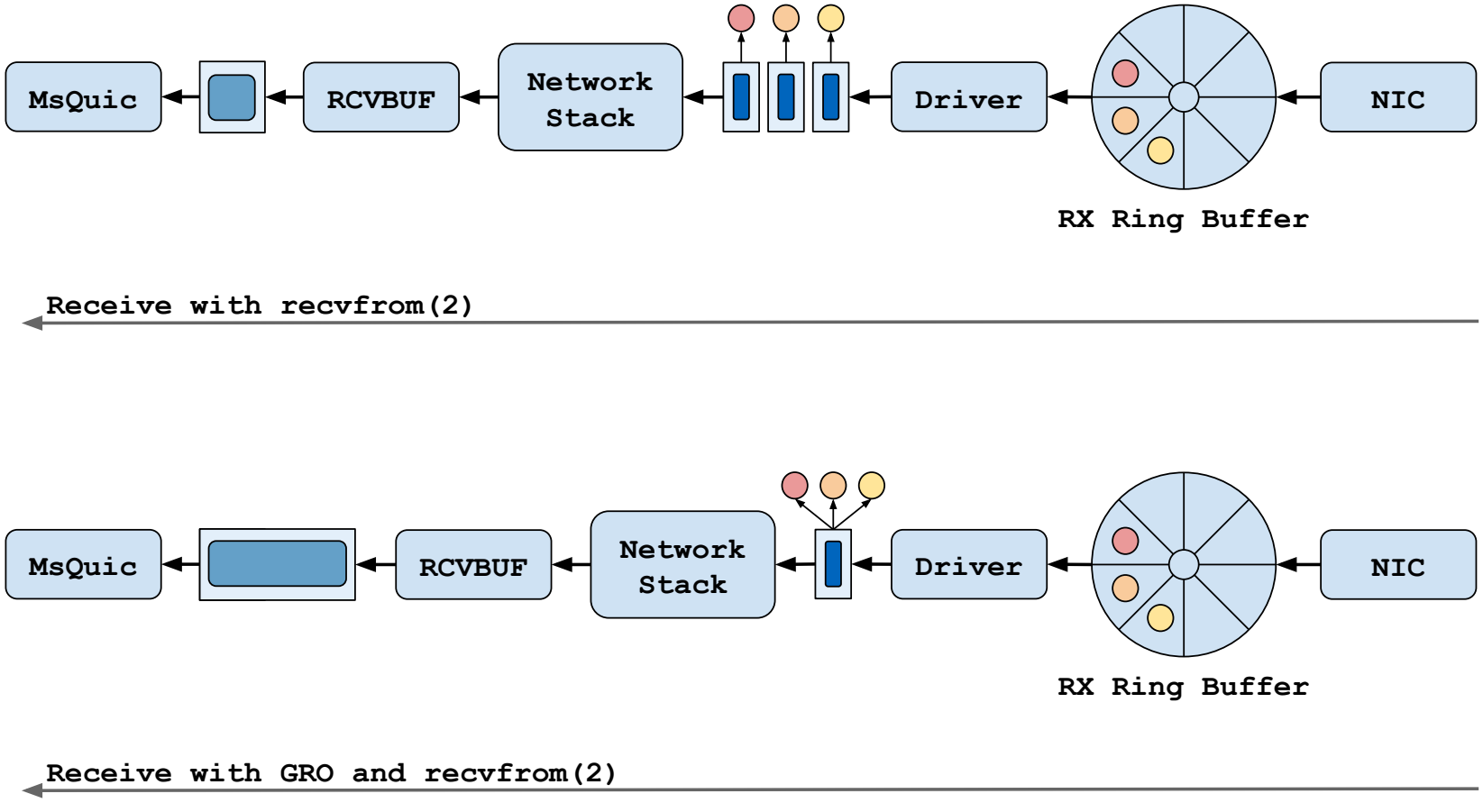
Additional Optimizations

recvmmsg(2)



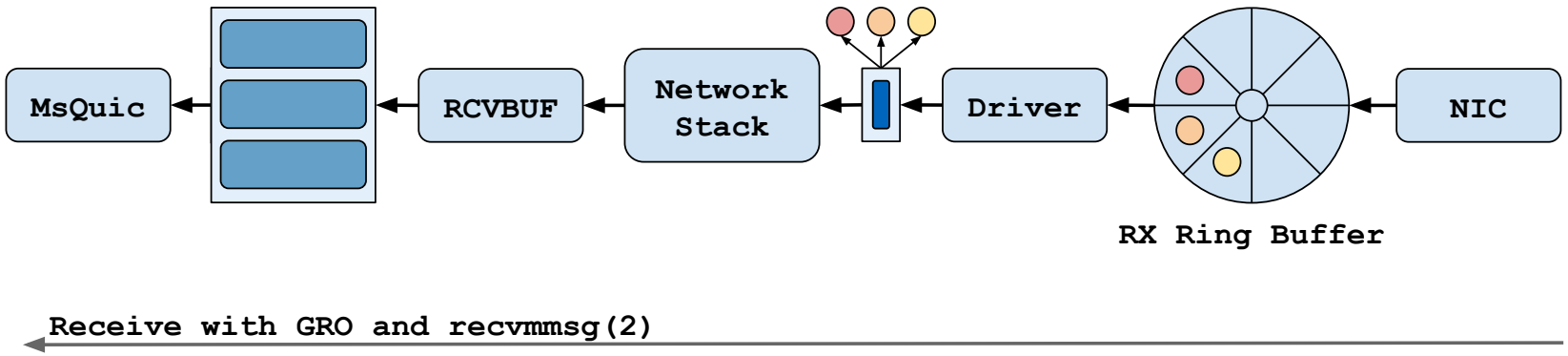
Additional Optimizations

Generic Receive Offload (GRO)



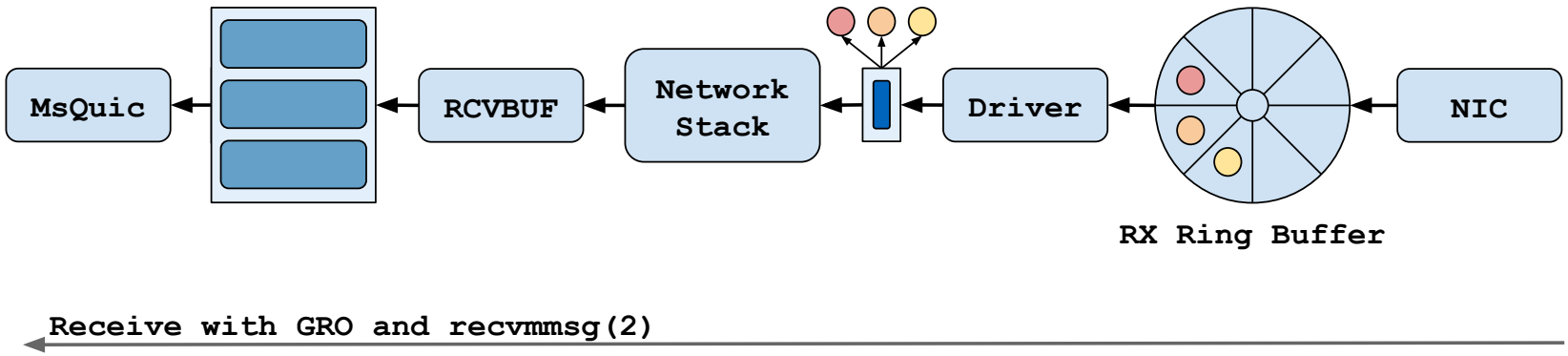
Additional Optimizations

GRO + recvmmsg(2)



Additional Optimizations

GRO + recvmmsg(2)



- A similar optimization exists for sending:
  - Generic Segmentation Offload (GSO)
  - sendmmsg(2)

Approach

Goodput

- Patch MsQuic[4] to support AF_XDP
- Run goodput tests in a client-server topology
- 20 iterations with 8GB file size using weak (Dogecoin) and strong (Opulous) CPU setups:
  - Dogecoin: Intel Xeon D-1518 @ 2.20GHz 4-core CPU, 32 GB of RAM, and Intel X552 10 GbE SFP+ NICs, over a 10G link[5]
  - Opulous: AMD EPYC 9354 @ 3.25 GHz 32-core CPU, 768 GB of RAM, and Mellanox ConnectX-6 NICs over a 100G link[6]



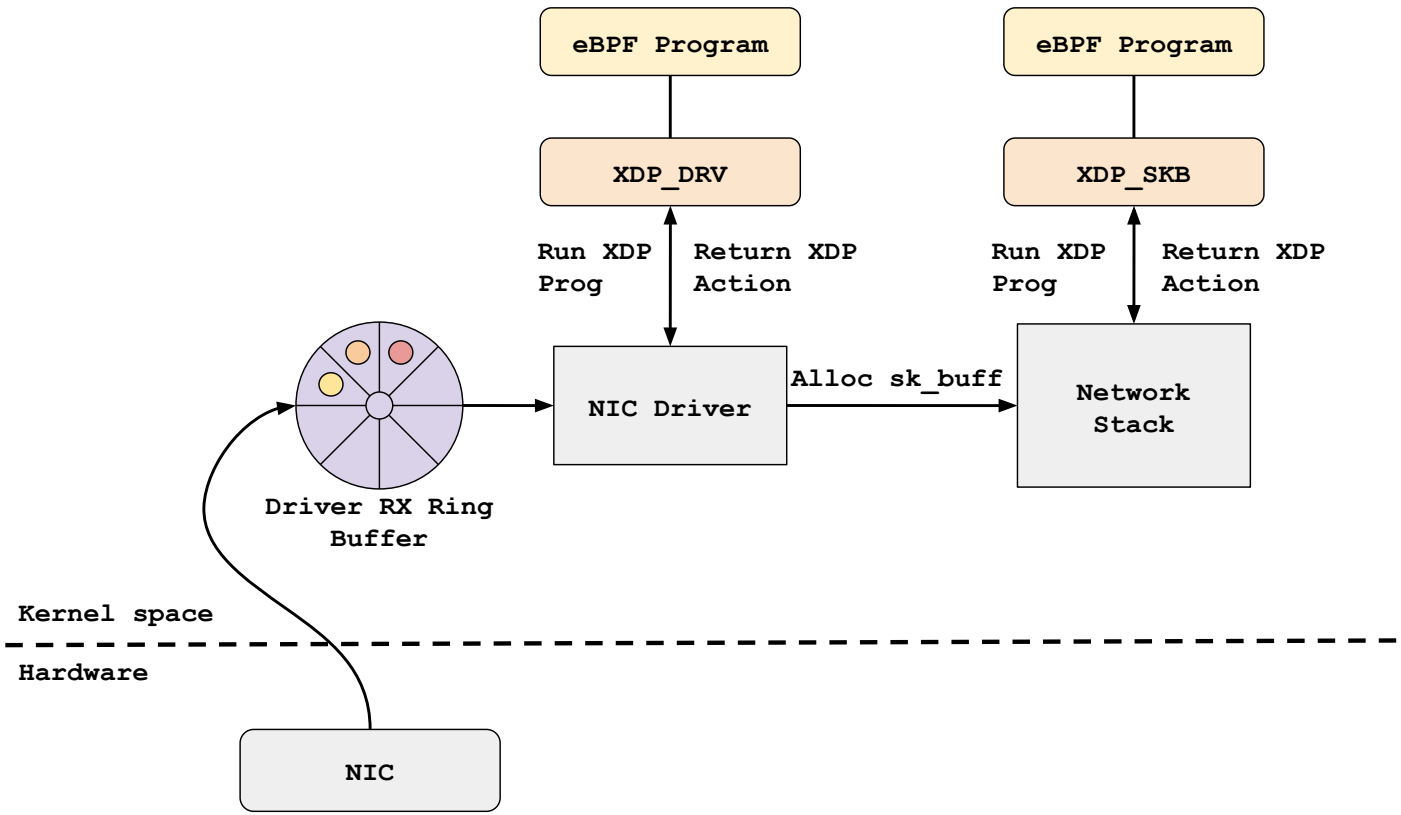
[4] https://github.com/microsoft/msquic

[5] Additionally, ixgbe driver (version 6.1.0-17-amd64) was used

[6] Additionally, mlx5_core driver (version 6.1.0-17-amd64) was used

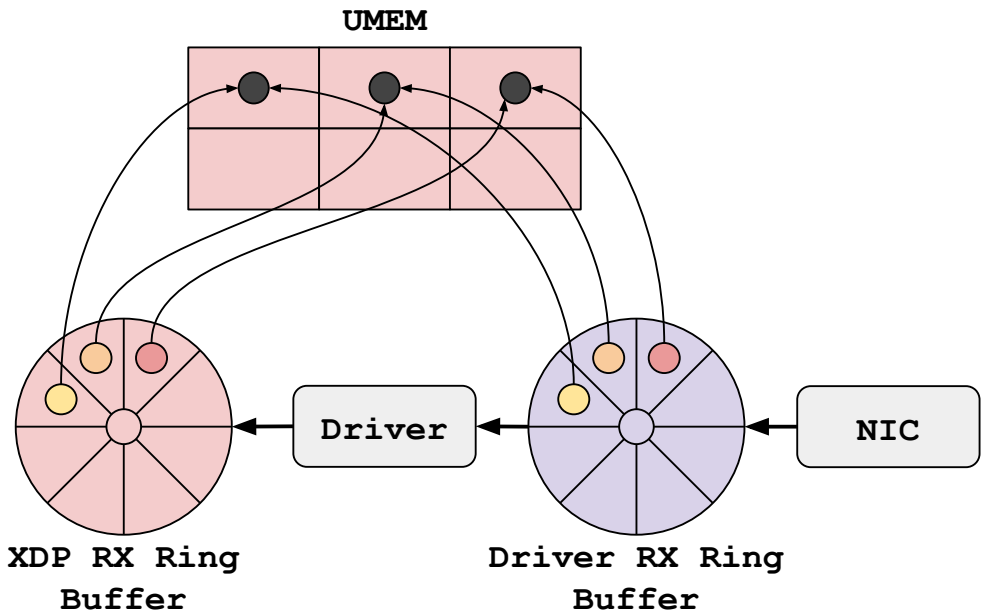
XDP Hook Points

- Two different XDP hook points: `XDP_DRV` and `XDP_SKB`
- `XDP_SKB` requires additional processing over `XDP_DRV`



XDP Types

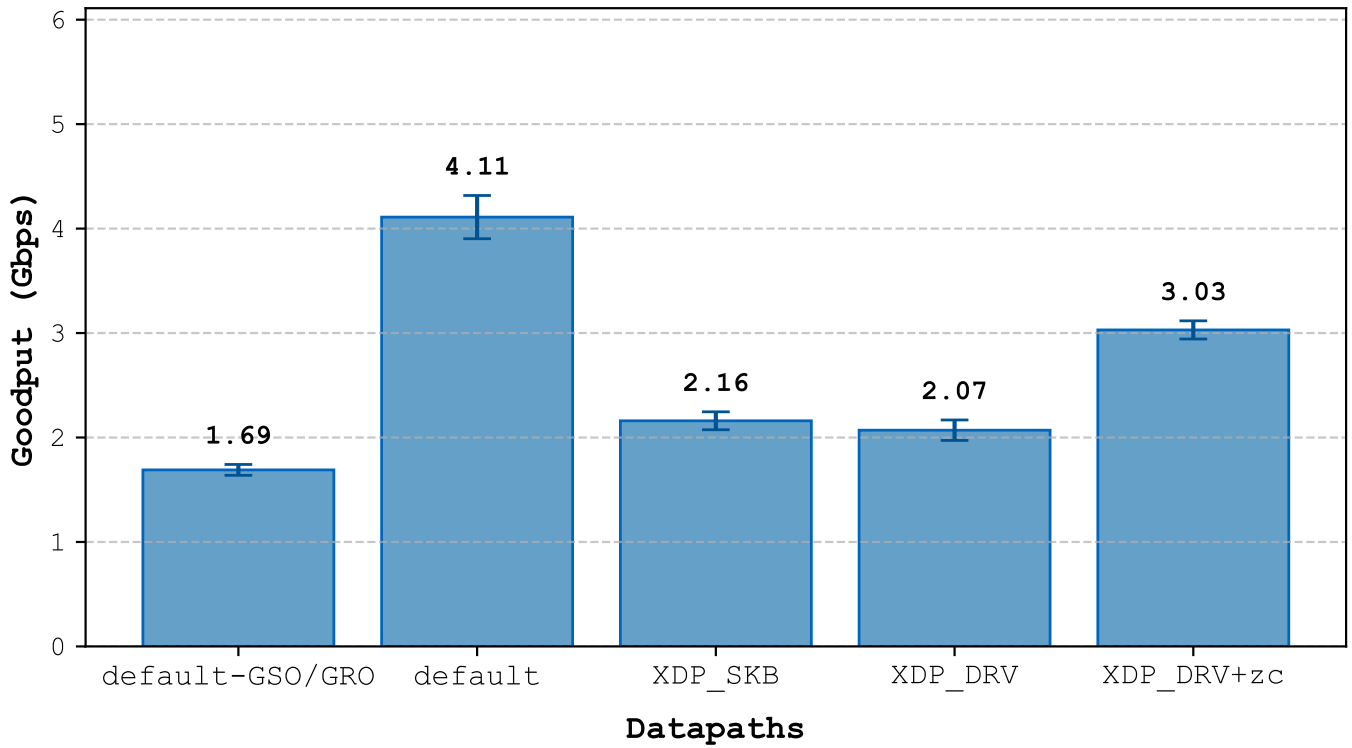
- Three XDP types: `XDP_SKB`, `XDP_DRV`, and `XDP_DRV+zc` (i.e. zero-copy)
- For `XDP_DRV+zc`, packet is copied by NIC directly into UMEM region
- For `XDP_SKB` and `XDP_DRV`, an intermediate copy into kernel memory is required
- A similar process exists for sending



Measurements

AF_XDP on the Server (Dogecoin)

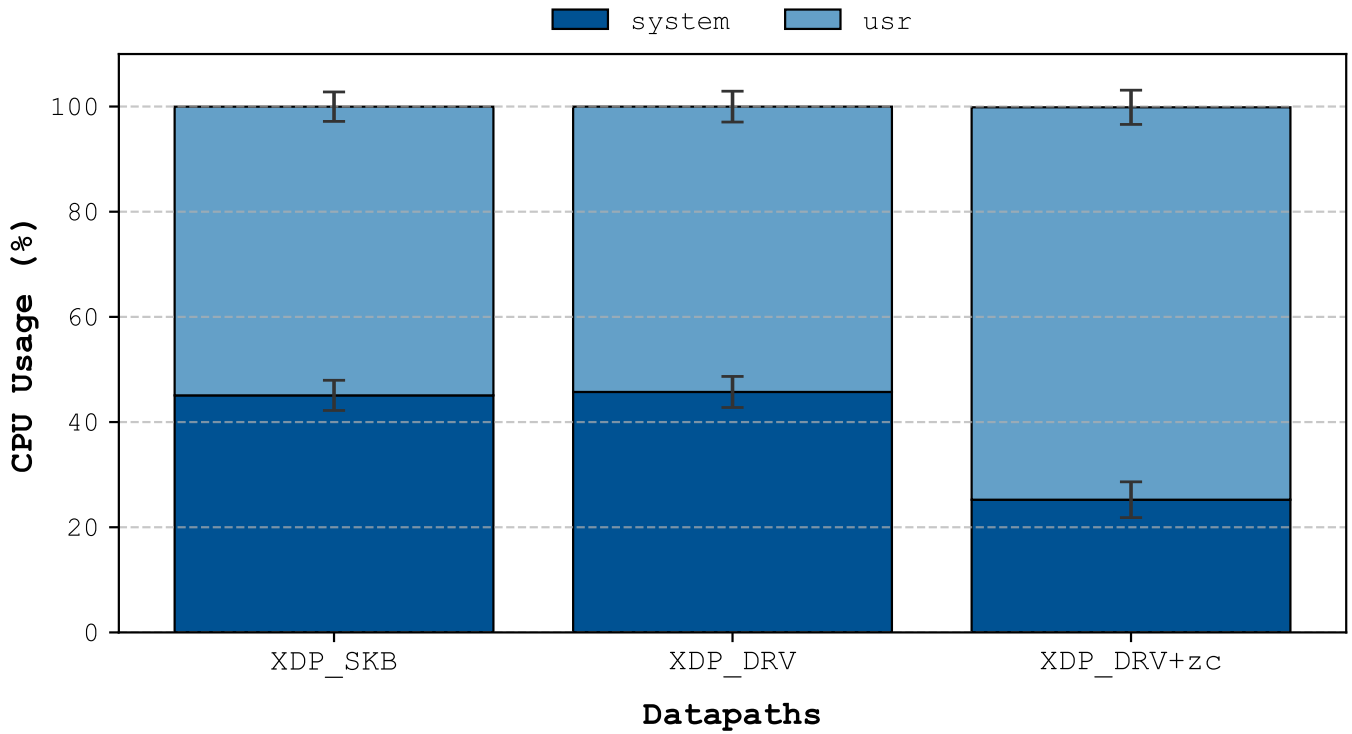
- `XDP_DRV+zc` exhibits higher goodput than substitutes but remains lower than `sendmmsg+GSO`
- `GSO` makes the difference within the default datapath



Measurements

AF_XDP on the Server, continued

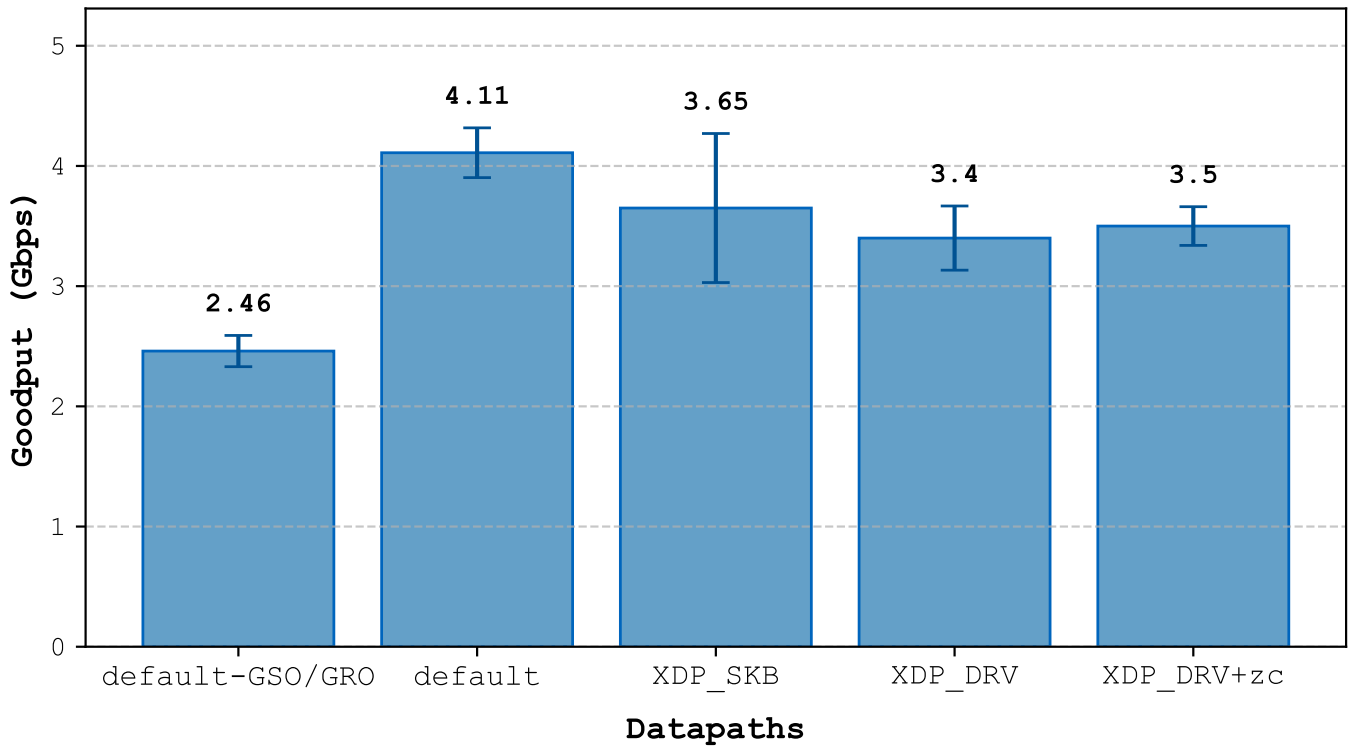
- System CPU usage for `XDP_SKB` and `XDP_DRV` higher than `XDP_DRV+zc`
- We used a perf trace to pinpoint the increased time spent copying data from the UMEM to kernel memory



Measurements

AF_XDP on the Client (Dogecoin)

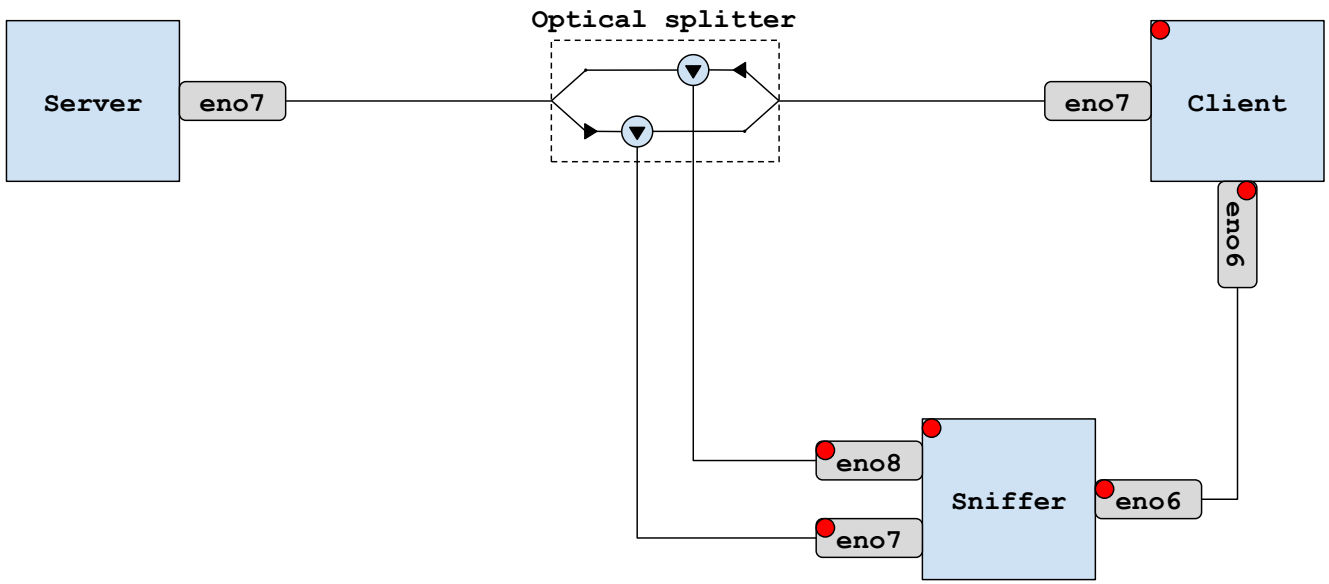
- Surprisingly, the XDP alternatives are no longer statistically significant from each other
- Yet, XDP modes are closer in goodput to the default datapath with all optimizations



Approach

NIC-to-Userspace Packet Traversal Time

- Sync NICs and system times using Precision Time Protocol (PTP)[7]
- 1 iteration with 128MB file size using Dogecoin[8]
- Generates more than 100k NIC-to-Userspace packet time differences
- Poor documentation and integration into QIR was a difficult and lengthy process

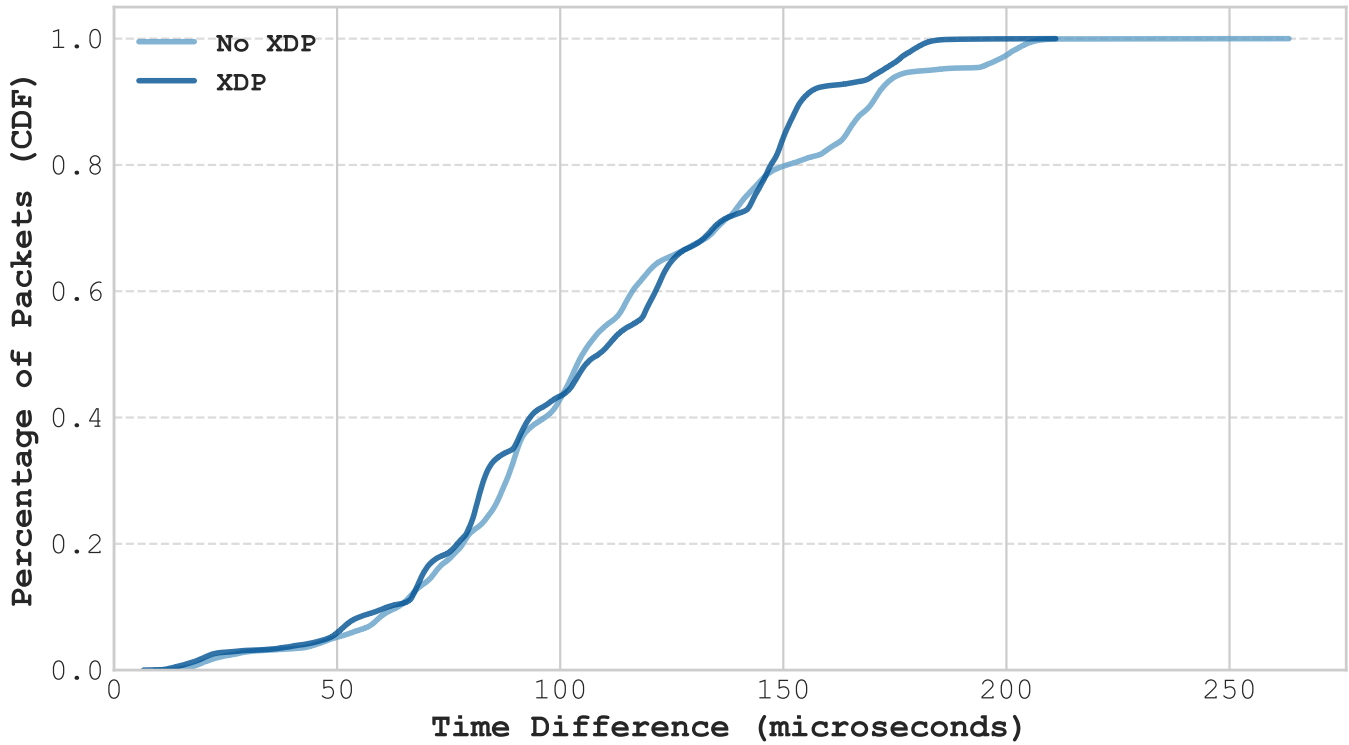


[7] https://github.com/richardcochran/linuxptp

[8] Dogecoin: Intel Xeon D-1518 @ 2.20GHz 4-core CPU, 32 GB of RAM, and Intel X552 10 GbE SFP+ NICs, using ixgbe driver (version 6.1.0-17-amd64) over a 10 Gbase link.

Measurements

- XDP and default datapath NIC-to-userspace times mostly remain in parity
- The long tail in the non-XDP case is likely due to additional queues in the receive path



Related Work

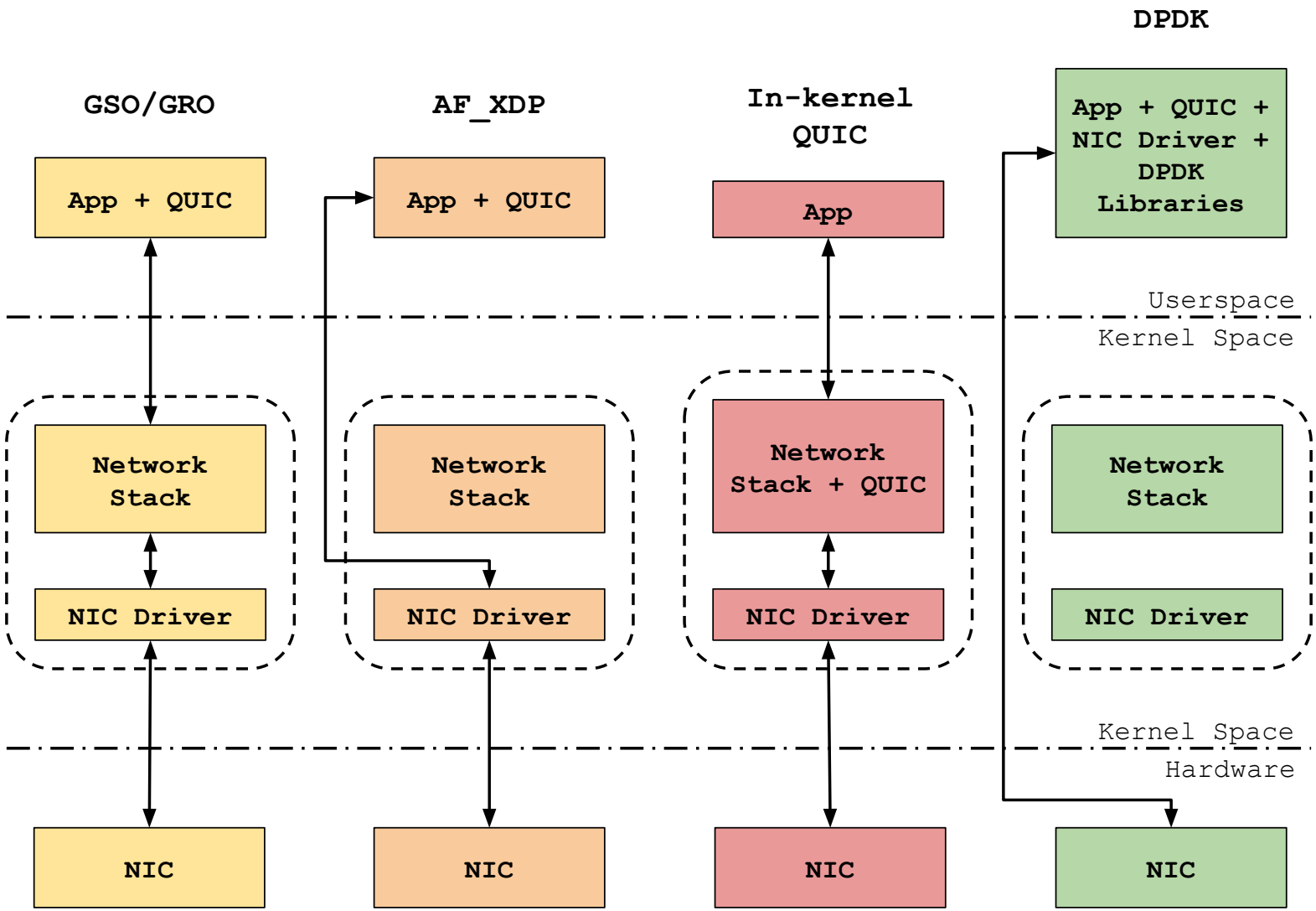
- *QUIC Performance Improvements Using Socket Optimizations and Kernel Bypass*[9] investigates methods to improve QUIC performance by means of patching MsQuic with DPDK and using socket optimizations
- lxin-quic[10] brings QUIC into the kernel, reducing context switches and copying between kernel and userspace
- DPDK[11] enables kernel bypass and allows application to connect directly to the NIC, reducing kernel overhead

[9] Späth, J. QUIC performance improvements using socket optimizations and kernel bypass. Master's thesis, TU Munich, 2023.

[10] https://github.com/lxin/quic

[11] https://www.dpdk.org/

QUIC Datapaths



Taxonomy of QUIC Datapaths

Metric	GSO/GRO	AF_XDP	In-kernel QUIC	DPDK
Standardized APIs	✓	×	✓	×
Kernel Integration	High	Medium	High	Low
NIC Sharing	✓	✓	✓	×
Hardware Agnostic	High	High*	High	Low
Maturity	High	Medium	Low	High

Conclusion

- To what extent can we increase QUIC performance by using XDP in a trivial way? Is there optimization potential?
  - AF_XDP is not able to achieve a higher goodput over the optimized default datapath
  - Also, AF_XDP NIC-to-userspace times remain consistent with the default datapath

- Can a taxonomy be created for QUIC optimizations, categorized by:
  - Implementation complexity
  - Performance improvement potential
  - Unwanted side effects
  - Privilege level usage
  - Deployment

✓ Yes, each datapath offers its unique set of tradeoffs

Future Work

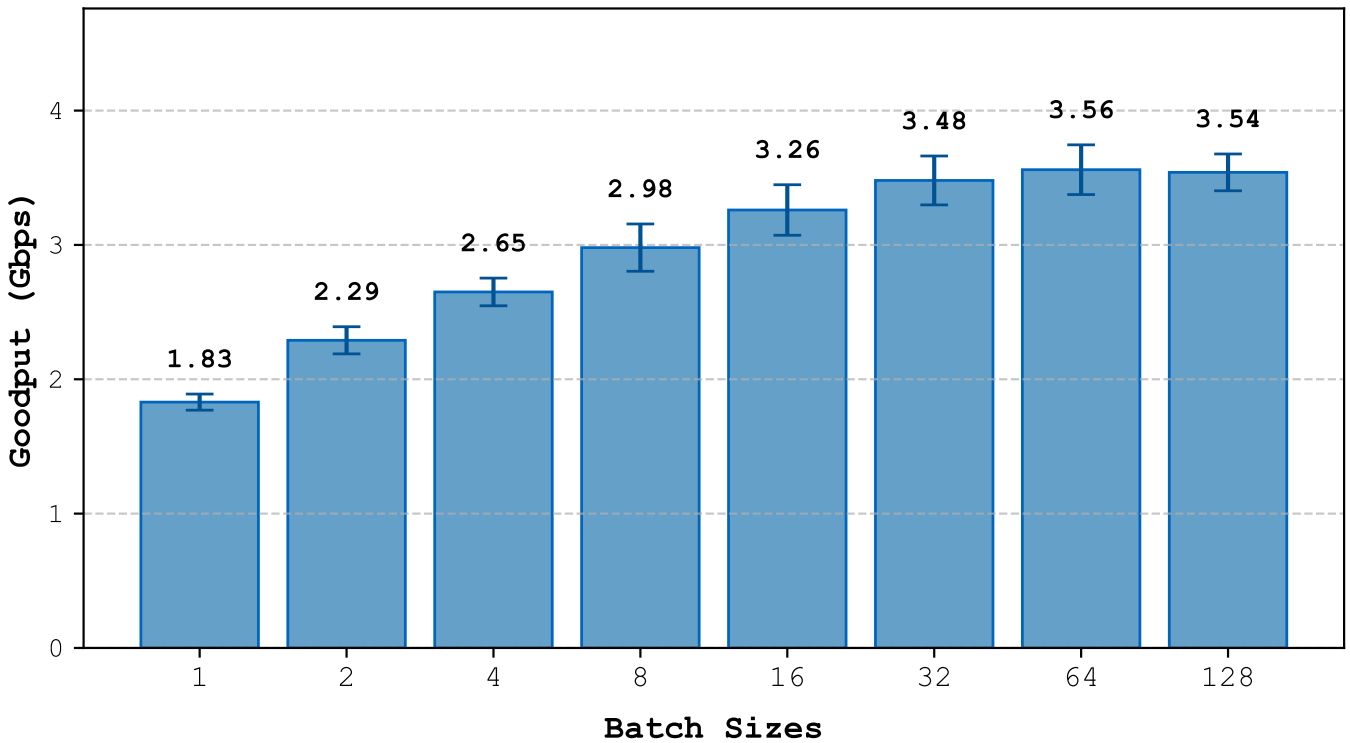
- Enable GRO for AF_XDP, for which driver support has been increasing [8]
- Expanding hardware offloads to include encryption[12]
- Perform goodput and latency measurements across all QUIC datapaths on the same testbed

[12] https://github.com/microsoft/net-offloads/blob/main/quic-encryption-offload.md

Appendix

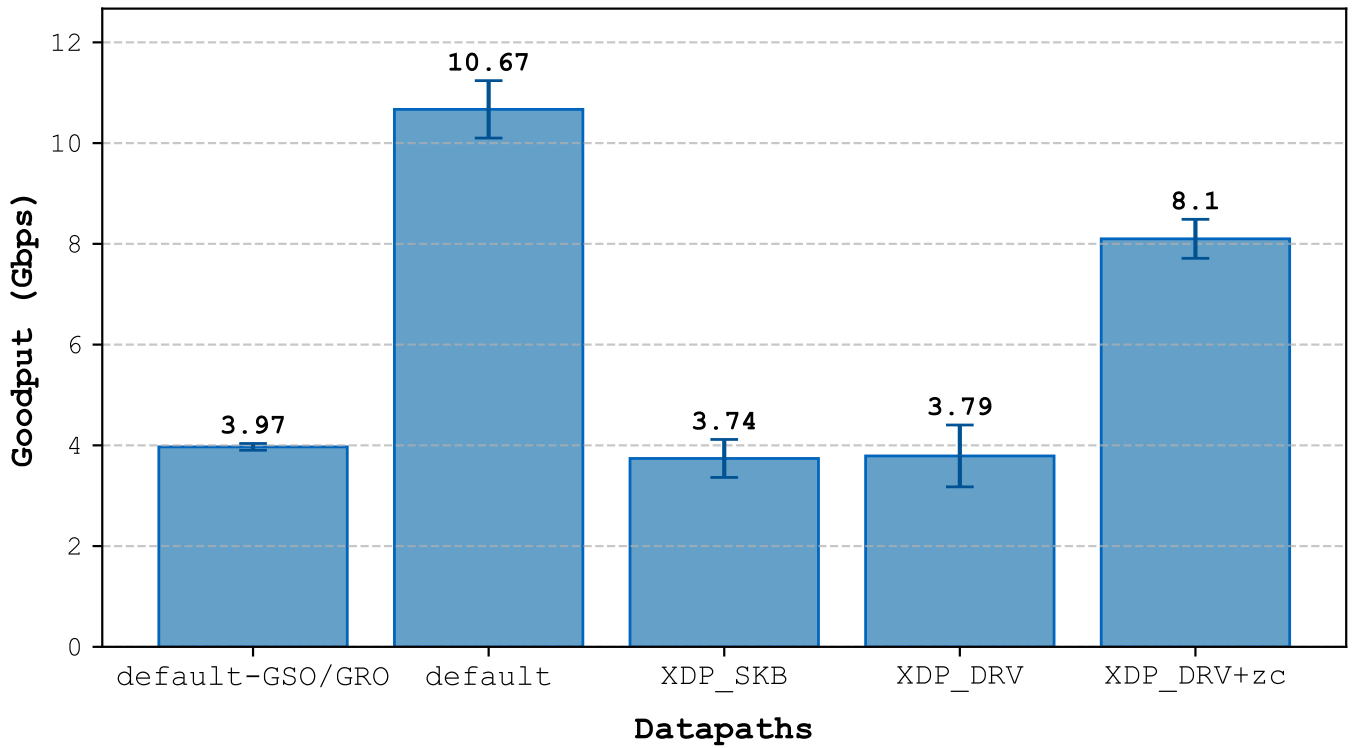
RX Batch Sizes with AF_XDP on the Client (Dogecoin)

- Goodput increases as RX batch size increases until 32
- The bottleneck changes; MsQuic must no longer wait for packets and instead spends its time on processing



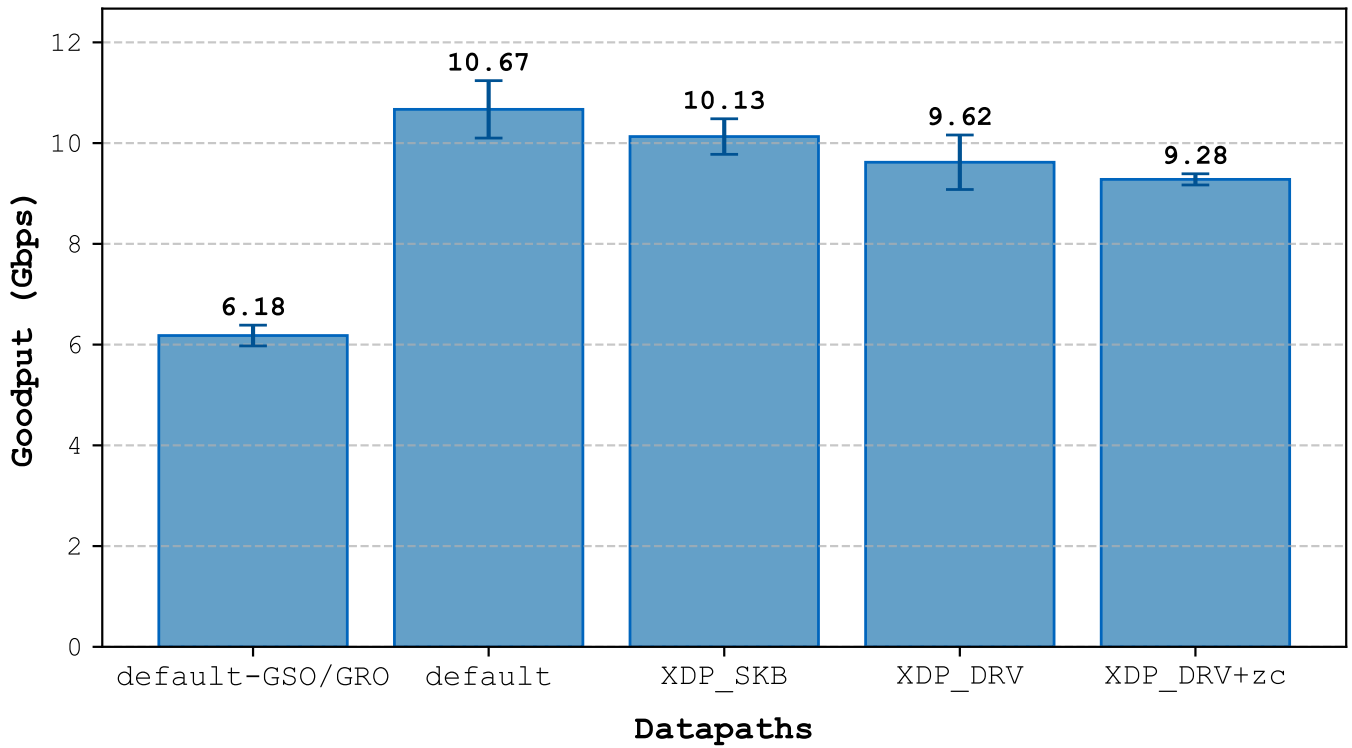
Appendix

AF_XDP on the Server (Opulous)



Appendix

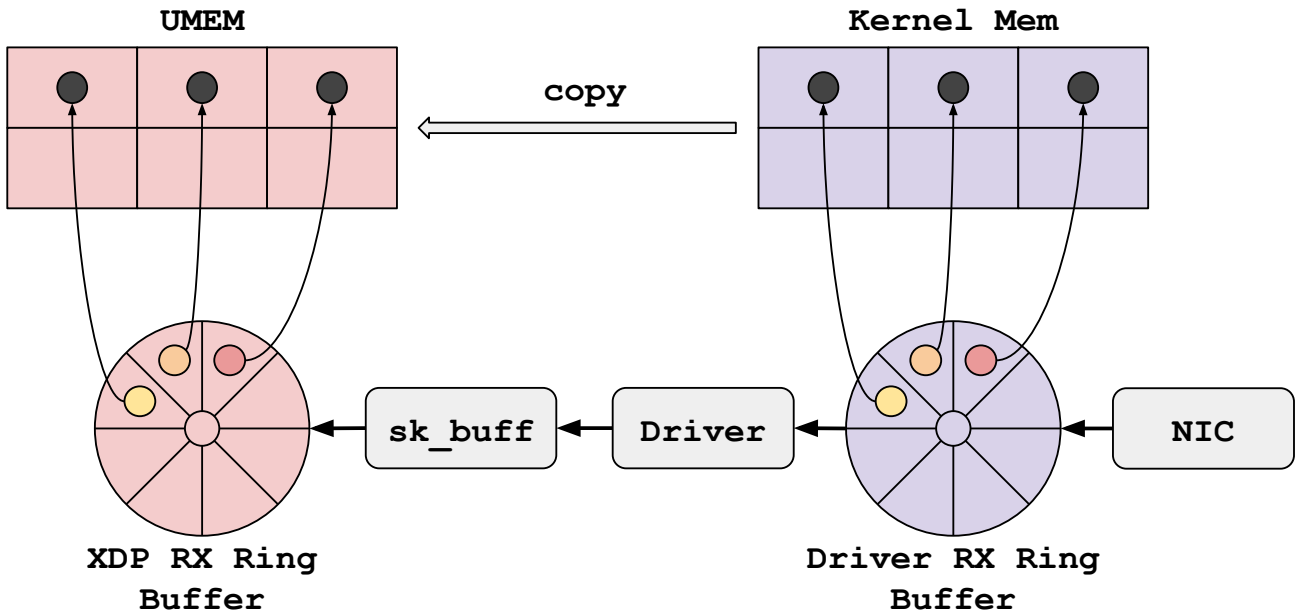
AF_XDP on the Client (Opulous)



Appendix

XDP Types: XDP_SKB

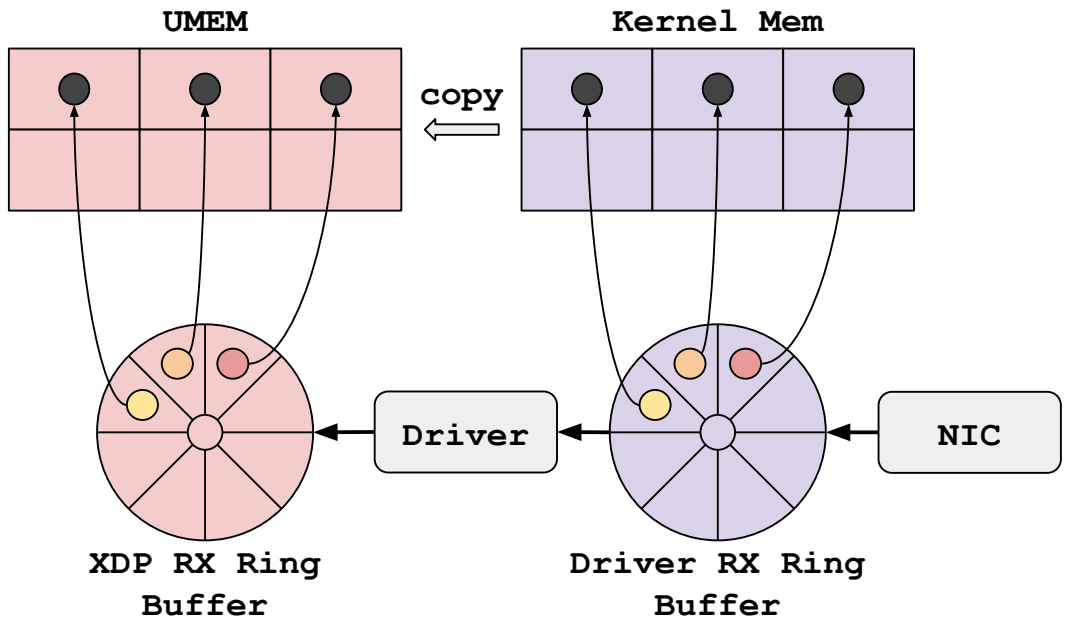
- Packet is copied by NIC into kernel memory and an sk_buff is allocated
- Packet is copied from kernel memory into the UMEM region



Appendix

XDP Types: XDP_DRV

- Packet is copied by NIC into kernel memory; no sk_buff this time
- Packet is copied from kernel memory into the UMEM region



Bibliography

[1] Cloudflare.
Cloudflare radar year in review 2024, 2024.
Accessed: 2024-12-14.

[2] Xumiao Zhang, Shuowei Jin, Yi He, Ahmad Hassan, Z. Morley Mao, Feng Qian, and Zhi-Li Zhang.
Quic is not quick enough over fast internet.
In *Proceedings of the ACM Web Conference 2024*, WWW '24, page 2713–2722. ACM, May 2024.

[3] Benedikt Jaeger, Johannes Zirngibl, Marcel Kempf, Kevin Ploch, and Georg Carle.
Quic on the highway: Evaluating performance on high-rate links.
pages 1–9, 06 2023.

[4] Marten Gartner.
Recapitulating af_xdp.
`https://medium.com/high-performance-network-programming/recapitulating-af-xdp-ef6c1ebead8`, 2022.
Figure: "The flow of packets using a normal socket and AF_XDP", retrieved from the article.

[5] Johannes Späth.
Quic performance improvements using socket optimizations and kernel bypass.
Master's thesis, Technical University of Munich, Garching, Bavaria, 2023.

[6] Xin Long.
In-kernel quic implementation with userspace handshake.
`https://github.com/lxin/quic`, 2023.
Accessed: 2025-04-20.

[7] Linux Foundation.
Data plane development kit (DPDK), 2015.

Bibliography

[8] Lorenzo Bianconi, Eelco Chaudron, Jesper Dangaard Brouer, and Toke Høiland-Jørgensen.
Bringing tso/gro and jumbo frames to xdp.
Linux Plumbers Conference, 2021.
Contribution to Linux Plumbers Conference 2021.